Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

Team of 3 · 6 build days · Hybrid delivery: local Docker (generator, Kafka, Airbyte, Airflow, FastAPI, React) + cloud (Amazon S3, Snowflake)

Problem

Fleet managers cannot combine vehicle diagnostics, position and cargo temperature in one place, so route deviations, refrigeration breaches and excessive idling are hard to investigate consistently.

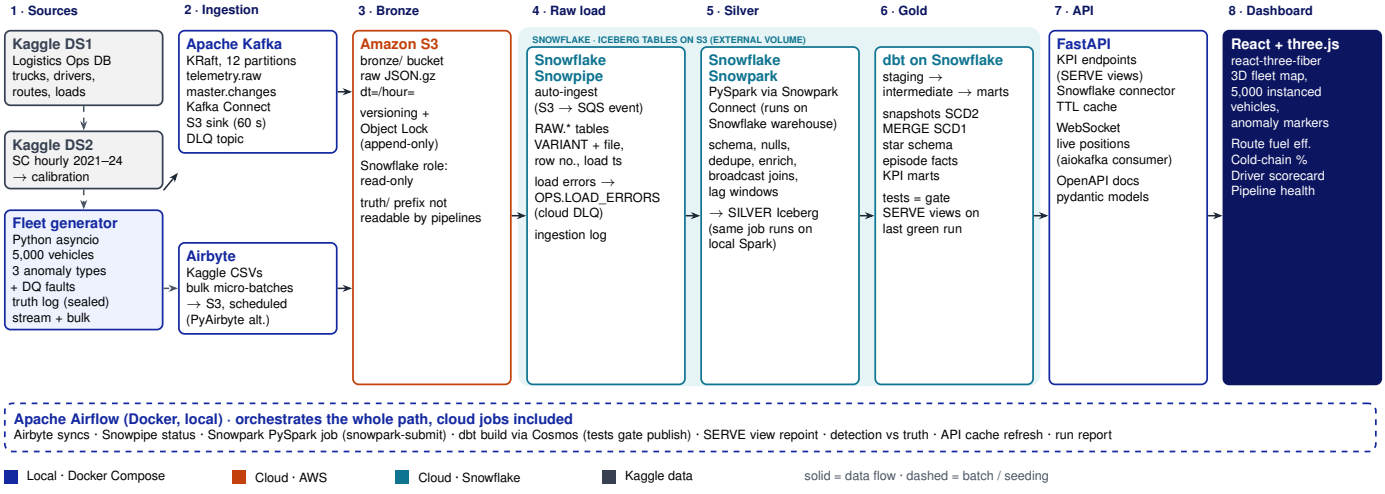
Objective

Build a telematics lakehouse and star-schema warehouse for **5,000 simulated delivery vehicles** on fixed corridors, serving route fuel efficiency, cold-chain compliance % and driver safety/idling scorecards in a 3D web dashboard.

Measurable targets

5,000 distinct vehicles · 0 duplicate keys in silver · Kafka = S3 = RAW + load errors · detection recall $\geq$ 95% / precision $\geq$ 90% vs injected truth · dashboard = API = SQL $\pm$0.1% · event $\rightarrow$ KPI $\leq$ 15 min.

End-to-end flow



Data work: how the Kaggle data and the generator connect

Source	Role
DS1 Logistics Ops DB (2022–24)	**Master data.** Seeds dim_vehicle, dim_driver, dim_route, dim_facility, dim_cargo; DS1 routes become fixed corridors; DS1 loads become trips; fuel purchases benchmark simulated km/L.
DS2 Supply-chain hourly (2021–24)	**Calibration + history.** Fits hour-of-day speed, fuel-rate, temperature-noise and driver-profile distributions into `calibration.json`; kept as a historical context fact.
Generator (asyncio + Faker)	Expands DS1 to **5,000 vehicles** (`VH-00001...` keeps `source_truck_id`); emits GPS, OBD-II fuel rate, RPM, engine temp and reefer temp per ping; stream to Kafka + hourly bulk files for Airbyte; seeded config.

Anomaly	Injected $\rightarrow$ detected (no labels read)
Cold-chain breach	Reefer +4–10 ℃ for 15–90 min $\rightarrow$ temp $>$ setpoint + 3 ℃ for $\geq$ 15 min
Excessive idling	Engine on, stopped 12–60 min $\rightarrow$ speed $<$ 2, RPM 500–1000 $\geq$ 10 min, off-facility
Route deviation	3–15 km off corridor $\rightarrow$ $>$ 2 km from polyline for $\geq$ 3 pings
DQ faults	Malformed, nulls, duplicates, late $\rightarrow$ Kafka DLQ / load errors, flags, dedupe, MERGE

Injections go to a sealed S3 truth prefix that pipelines cannot read; a harness scores detections against it. Units converted to metric in dbt staging.

KPIs (documented definitions)

KPI	Definition
Fuel efficiency / route	Σ km $\div$ Σ litres per route (km/L); km = haversine between pings, litres = fuel_rate $\times$ Δt
Cold-chain compliance	Reefer minutes within setpoint $-2/+3$ ℃ $\div$ reefer-active minutes on cold loads (simulation threshold, not certification)
Driver scorecard	Idle % = idle min $\div$ engine-on min; safety score = 100 $-$ (5·harsh/100 km + 3·speeding/100 km + 0.5·idle %), 0–100

Design by layer

Ingestion & bronze

- Kafka topics `telemetry.raw` (12 partitions by vehicle_id), `master.changes` ; Kafka Connect S3 sink with native DLQ topic.
- Airbyte lands Kaggle files and bulk micro-batches on a schedule.
- S3 bronze: raw JSON.gz by dt/hour, versioning + Object Lock. Snowflake Snowpipe auto-ingests into RAW; load errors = cloud DLQ.

Snowflake: Snowpark silver + dbt gold

- **Snowflake Snowpark** runs our PySpark job on a Snowflake warehouse via Snowpark Connect (same script on local Spark for tests): schema, nulls, dedupe, broadcast enrichment, lag windows → SILVER Iceberg.
- dbt: staging → intermediate → marts; snapshots SCD2, MERGE SCD1; star schema on Snowflake-managed Iceberg tables.
- dbt tests gate publish: SERVE views point to the last green run.

API & dashboard

- FastAPI: KPI endpoints on SERVE views with TTL cache; WebSocket of live positions from Kafka.
- React + three.js (react-three-fiber): 3D fleet map with 5,000 instanced vehicles and anomaly markers.
- Pages: fuel efficiency, cold-chain, driver scorecard, pipeline health.

Tools and alternatives

Layer	Chosen	Runs	Alternatives
Generator	Python 3.11 asyncio, aiokafka, Faker, numpy	Local	threading/multiprocessing, Mimesis, SDV
Streaming	Apache Kafka (KRaft) + Kafka Connect S3 sink	Local	Redpanda, Confluent Cloud, Amazon MSK
Batch ingestion	Airbyte OSS / PyAirbyte	Local	Airbyte Cloud, Apache NiFi, Fivetran
Bronze	Amazon S3 (versioning, Object Lock)	AWS	MinIO, ADLS, GCS
Raw load	Snowflake Snowpipe (auto-ingest from S3)	Snowflake	COPY INTO, external tables, Airbyte → Snowflake
Processing	Snowflake Snowpark (PySpark via Snowpark Connect)	Snowflake	Snowpark Python API, EMR / Databricks Spark
Modeling	dbt Core on Snowflake (dbt-snowflake, dbt_utils, dbt_expectations)	Snowflake	dbt Cloud, Dynamic Tables, SQLMesh
Lakehouse / SQL	Snowflake Iceberg tables on S3 + Snowflake SQL	Snowflake	Native Snowflake tables, Trino/Athena on Iceberg
Quality	dbt tests + Snowflake Data Metric Functions	Snowflake	Great Expectations, Soda Core
Orchestration	Apache Airflow + astronomer-cosmos	Local	MWAA, Dagster, Prefect
API / dashboard	FastAPI; React + Vite + TS, three.js (r3f), Recharts	Local	Flask; deck.gl / Mapbox; Superset as fallback
Infra / dev	Docker Compose, Terraform (AWS + Snowflake), GitHub CI	Local	AWS CDK, schemachange, GitLab CI

Work distribution: three parallel end-to-end slices

Each member owns **one KPI slice through every layer** (data creation → ingestion → silver → gold → API → dashboard) plus **one shared platform piece**. All three work on the same layer on the same day, so load is equal and nobody waits on another member's layer.

Layer	Member A · Cold-chain slice	Member B · Route & fuel slice	Member C · Driver safety slice
Shared platform (Day 1)	Generator core (async shards, config, sinks, truth log); Kafka + Connect S3 sink + DLQ; S3 bronze + Object Lock	Snowflake roles, warehouses, external volume; Snowflake Snowpipe + load errors; Snowpark job framework; dbt project skeleton	Docker Compose + CI; Airflow DAG skeleton + Cosmos; FastAPI shell; React shell + layout; pipeline health page
Data creation (generator module)	Reefer & cargo signals: setpoints from DS1 loads, DS2 temperature noise	GPS, corridors from DS1 routes, speed + fuel rate from DS2 traffic/fuel	RPM, engine temp, decel, idling; DS2 driver profiles; `master.changes` events
Anomaly: injects → detects	injects route deviation · detects cold-chain breach	injects idling · detects route deviation	injects cold-chain breach · detects idling
Ingestion	Airbyte: DS1 loads, customers, facilities; Kafka DLQ tests	Airbyte: DS1 routes, trucks, fuel purchases + DS2	Airbyte: DS1 drivers, safety incidents; bulk micro-batches
Silver (Snowflake Snowpark)	Reefer features, sensor-gap flags, rolling 15-min average	Haversine + fuel deltas, corridor distance (broadcast)	Engine-on / idle flags, harsh-brake + speeding flags
Gold (dbt)	dim_cargo, dim_customer, fact_coldchain_excursion, cold-chain KPI mart	dim_vehicle (SCD2), dim_route (SCD1), fact_telemetry_ping, fact_trip, fact_route_deviation, fuel KPI mart	dim_driver (SCD2), fact_idle_episode, fact_driver_daily, scorecard mart
API (FastAPI)	`/kpi/cold-chain`, `/trips/{id}/temperature`	`/kpi/fuel-efficiency`, `/ws/positions`	`/kpi/drivers`, `/health/pipeline`
Dashboard (React)	Cold-chain page + temperature timeline	3D fleet map (three.js) + route page	Driver scorecard page + health page
Tests & evidence	Own slice tests; reconciles B's KPI	Own slice tests; reconciles C's KPI	Own slice tests; reconciles A's KPI

6-day timeline: all three in parallel on the same layer

Day 1 · Gate 1
Each builds a platform piece; profile own DS1 tables; freeze contract, silver schema and API models. *Exit:* event reaches S3.

Day 2 · Gate 2
Each: generator module, Airbyte syncs, Snowpipe → RAW, Snowpark features for own slice. *Exit:* 5,000 vehicles in silver.

Day 3 · Core I
Each: injected anomaly, dbt dims + facts, detection rule for assigned anomaly, API stub.

Day 4 · Gate 3
Each: KPI mart + tests, API endpoint, React page; full DAG run. *Exit:* raw → dashboard.

Day 5 · Gate 4
Each: own slice tests + cross-reconciles another slice's KPI; failure, SCD, time-travel tests.

Day 6 · Gate 5 prep
Freeze 12:00; final run; each: own evidence + deck section; 2 rehearsals.

Handbook fit: All loading, processing and warehousing run in Snowflake: Snowpipe loads, Snowpark runs our PySpark code (Pillar 3), and Snowflake Iceberg tables cover Pillar 4. Airflow stays because Pillar 5 requires it. Interfaces frozen on Day 1 so slices never block each other; daily 17:00 integration; each PR reviewed by another slice owner; XS warehouses with auto-suspend and a resource monitor cap credits.